1 **Design & Implementation of Human-Computer Interfaces**
NPTEL-MOOCS

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

2 **L27 - Black Box Testing**

Dr Samit Bhattacharya
Computer Science and Engineering
IIT Guwahati

3

4 **Recap**

- Code testing - TWO ways
  - Review-based
  - Execution-based

5 **Recap**

- Review based testing - TWO types
  - Code walkthrough (similar to cognitive walkthrough)
  - Code inspection (similar to heuristic evaluation)

6 **Review Based Testing – Pros & Cons**

- Review-based testing informal – mostly evaluates code qualitatively
- Good for early evaluation to 'clean up' the code before more rigorous and formal testing is done

7 **Next**

- Let's learn about the other code testing method
  - Execution based testing
  - Also can be thought of as (formal) program testing

8 **(Formal) Program Testing**

- Consists of
  - Providing program a set of test inputs (or test cases)
  - Observing program behavior (and/or output)

9 **(Formal) Program Testing**

- If program fails to behave as expected (or output mismatch), the conditions under which failure occurs are noted for later debugging and correction

10 **Terminology**

- Test case – a triplet [I,S,O]
  - I = data input
  - S = system state at input time
  - O = expected output
- For simplicity, we will consider only the doublet [I,O]

11 **Terminology**

- Test suite - set of all test cases with which a given software is to be tested

12 **Choice of Test Cases (Suite)**

- Exhaustive testing impractical - possible input data values extremely large or infinite
  - We must design test suite that is of reasonable size and can uncover as many errors existing in the system as possible

13 **Choice of Test Cases (Suite)**

- Randomly selected test cases not necessarily contribute to significance of test suite
  - Need not detect additional defects not already detected by other test cases

14 **Choice of Test Cases (Suite)**

- Number of test cases not indication of testing effectiveness
- Large number of test cases selected at random does not guarantee all (or even most) of the errors will be uncovered

15 **Choice of Test Cases (Suite)**

- Implication - test suite should be carefully designed (not decided randomly)
- Require systematic approaches

16 **(Systematic) Code Testing**

- Broadly of TWO types

17 **(Systematic) Code Testing**

- Functional testing - test cases designed using only functional specification of software, i.e. without any knowledge of internal structure [Black-box testing]

18 **(Systematic) Code Testing**

- Structural testing - test cases designed using knowledge of internal structure of software [white-box testing]

19

Black-Box Testing

20 **Idea**

- Design test cases based on input/output values ONLY [no knowledge of design or code required]

21 **Idea**

- TWO main approaches to design test cases
  - Equivalence class partitioning
  - Boundary value analysis

22 **Equivalence Class & Partitioning**

- Domain of input values partitioned into sets – each called 'equivalence class'

23 **Equivalence Class & Partitioning**

- Program behavior *similar* for every input data belonging to an equivalence class

24 **Equivalence Class & Partitioning**

- Testing code with any ONE value of an equivalence class is as good as testing with ALL input values belonging to that class

25 **Boundary Value Analysis**

- Check at the class boundaries

26 **Note**

- In the next lecture, we shall learn more on black-box testing with examples

27 **Reference**

- Rajib Mall (2018). Fundamentals of Software Engineering, 5th ed, PHI Learning Pvt Ltd. Chapter 10
- Roger S Pressman (2020). Software Engineering: A Practitioner's Approach, 9th ed, McGraw-Hill Education, New York, Chapter 19-21